Сбалансированные деревья поиска

Теория

1. Бинарное дерево. Бинарное дерево поиска

Вопросы и задачи для самоподготовки

Бинарное дерево поиска

1. Что такое бинарное дерево поиска? Чем оно отличается от обычного бинарного дерева?
2. Операции и их сложность
   - Вставка
   - Удаление
   - Поиск элемента
   - Поиск минимума/максимума, предыдущего/следующего элемента
   - Повороты
   - Обходы в прямом (preorder), центрированном/имметричном (inorder) и обратном порядках (postorder) порядках
3. Выполнинте сл. упражения

12.1-1. Начертите бинарные деревья поиска высотой 2, 3, 4, 5 и 6 для множества ключей $\{1, 4, 5, 10, 16, 17, 21\}$.

12.2-1. Пусть у нас имеется ряд чисел от 1 до 1000, организованных в виде бинарного дерева поиска, и мы выполняем поиск числа 363. Какая из следующих последовательностей *не* может быть последовательностью проверяемых узлов?

а) 2, 252, 401, 398, 330, 344, 397, 363.
б) 924, 220, 911, 244, 898, 258, 362, 363.
в) 925, 202, 911, 240, 912, 245, 363.
г) 2, 399, 387, 219, 266, 382, 381, 278, 363.
д) 935, 278, 347, 621, 299, 392, 358, 363.

12.3-2. Предположим, что бинарное дерево поиска строится путем многократной вставки в дерево различных значений. Покажите, что количество узлов, проверяемых при поиске некоторого значения в дереве, на один больше, чем количество узлов, проверявшихся при вставке этого значения в дерево.

12.3-3. Отсортировать множество из n чисел можно следующим образом: сначала построить бинарное дерево поиска, содержащее эти числа (вызывая процедуру TREE_INSERT для вставки чисел в дерево одно за другим), а затем выполнить центрированный обход получившегося дерева. Чему равно время работы такого алгоритма в наилучшем и наихудшем случае?

12.3-5. Является ли операция удаления "коммутативной" в том смысле, что удаление x с последующим удалением y из бинарного дерева поиска приводит к тому же результирующему дереву, что и удаление y с последующим удалением x? Обоснуйте свой ответ.

13.2-2. Покажите, что в каждом бинарном дереве поиска с n узлами имеется ровно $n - 1$ возможных поворотов.

Красно-чёрные деревья

1. Свойства КЧ-дереьев
2. Операции и их сложность
   - Вставка
   - Удаление
   - Поиск элемента
3. Чему равна высота КЧ-дерева с n внутренними узлами? Почему?
4. Выполните сл. упражнения

13.1-1. Начертите в стиле рис. 13.1*a* полное бинарное дерево поиска высоты 3 с ключами $\{1, 2, \ldots, 15\}$. Добавьте к нему листья NIL и раскрасьте получившееся дерево разными способами, так чтобы в результате получились красно-черные деревья с черной высотой 2, 3 и 4.

13.1-3. Определим ***ослабленное красно-черное дерево*** как бинарное дерево поиска, которое удовлетворяет красно-черным свойствам 1, 3, 4 и 5. Другими словами, корень может быть как черным, так и красным. Рассмотрите ослабленное красно-черное дерево T, корень которого красный. Если мы перекрасим корень дерева T из красного цвета в черный, будет ли получившееся в результате дерево красно-черным?

13.1-5. Покажите, что самый длинный нисходящий путь от вершины x к листу красно-черного дерева имеет длину, не более чем в два раза превышающую кратчайший нисходящий путь от x к листу.

13.1-6. Чему равно наибольшее возможное число внутренних узлов в красно-черном дереве с черной высотой k? А наименьшее возможное число?

13.1-7. Опишите красно-черное дерево с n ключами с наибольшим возможным отношением количества красных внутренних узлов к количеству черных внутренних узлов. Чему равно это отношение? Какое дерево имеет наименьшее указанное отношение, и чему равна его величина?

13.3-1. В строке 16 процедуры RB_INSERT мы окрашиваем вновь вставленный узел в красный цвет. Заметим, что если бы мы окрашивали его в черный цвет, то свойство 4 не было бы нарушено. Так почему же мы не делаем этого?

13.3-2. Изобразите красно-черные деревья, которые образуются при последовательной вставке ключей 41, 38, 31, 12, 19 и 8 в изначально пустое красно-черное дерево.

13.3-5. Рассмотрим красно-черное дерево, образованное вставкой n узлов при помощи процедуры RB_INSERT. Докажите, что если $n > 1$, то в дереве имеется как минимум один красный узел.

AVL-дерево

1. Свойства AVL-дереьев
2. Операции и их сложность
   - Вставка
   - Удаление
   - Поиск элемента
3. Чему равна высота дерева с n ключами? Докажите.

B-дерево

1. Определение. Что такое характеристическое число T? За что оно отвечает?
2. Операции и их сложность
   - вставка

- поиск
- удаление

3. Теорема о высоте B-дерева
4. Выполните сл. упражнения

18.1-1. Почему минимальная степень B-дерева не может быть равна 1?

18.1-2. Для каких значений t дерево на рис. 18.1 является корректным B-деревом?

18.1-3. Изобразите все корректные B-деревья с минимальной степенью 2, представляющие множество $\{1, 2, 3, 4, 5\}$.

18.1-4. Чему равно максимальное количество ключей, которое может храниться в B-дереве высотой h, выраженное в виде функции от минимальной степени дерева t?

18.2-1. Покажите результат вставки ключей $F, S, Q, K, C, L, H, T, V, W, M, R, N, P, A, B, X, Y, D, Z, E$ в указанном порядке в изначально пустое B-дерево с минимальной степенью 2. Нарисуйте только конфигурации дерева непосредственно перед выполнением разбиения и окончательный вид дерева.

18.2-2. Поясните, при каких условиях могут выполняться излишние операции DISK_READ и DISK_WRITE в процессе работы процедуры B_TREE_INSERT (если таковые могут иметь место). Под излишней операцией чтения подразумевается чтение страницы, уже находящейся в оперативной памяти, а под излишней записью — запись на диск информации, идентичной уже имеющейся там.

18.2-3. Как найти минимальный ключ в B-дереве? Как найти элемент B-дерева, предшествующий данному?

PATRICIA

1. Определение. Связь с trie.
2. Осн. операции и их сложность
   - вставка
   - поиск
   - удаление
3. Максимальное к-во сравнений разрядов в среднем и в худшем случае при вставке или поиске?
4. Как получить все ключи, отсортированные в алфавитном порядке?

Код на защите

1. Операции с BST:
   - вставка
   - поиск
   - удаление
   - повороты
   - поиск минимума/максимума, предыдущего/следующего по неубыванию элемента
2. Операции с декартовыми деревьями:
   - split
   - merge

- вставка
- удаление

3. Операции с PATRICIA:
   - метод `Item* searchRecursive(Node* h, string key, int prevBit)` при условии наличия метода `digit(string key, int bit)`, который вычисляет бит (0 или 1) на позиции `bit`
4. Операции с B-деревом:
   - метод `Item* searchRecursive(Node* h, string key)`